Quiz #4 (Unit 4 - Extended): CPU Arithmetic, Pipelining & Hazards

Total Marks: 100.00

Hardware Arithmetic, Pipeline Hazards, Register Conventions & Algorithm Tracing

Name: ..... **Student ID:** ..... **Score:** / 100.00

> **Instructions:** Answer all 28 questions. Adhere strictly to question format tags: *[Multiple Choice]* (single choice), *[Select All That Apply]* (one or more options), or *[Fill in the Blank]* (exact strings or numbers). Calculators are not permitted.

PART 1: HARDWARE ARITHMETIC & FUNCTIONAL UNITS

Question 1

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Fundamental CPU datapath primitives: basic arithmetic building blocks integrated directly into simple ALU execution hardware.

Which of the following arithmetic operations are implemented directly as fundamental building blocks in primitive CPU execution hardware?

- ◯ a. MUL and DIV
- ◯ b. ADD and SUB
- ◯ c. EXP and SQRT
- ◯ d. MOD and REM

Question 2

Mark 2.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: ALU hardware sharing: how two's complement arithmetic allows subtractors to reuse adder circuits.

Which of the following statements regarding hardware implementation of addition and subtraction in a digital ALU are **TRUE**? (Select all that apply.)

- ☐ a. Hardware subtraction ($A - B$) is implemented using the adder by computing $A + (\sim B + 1)$.
- ☐ b. A dedicated subtractor circuit completely separate from the adder is strictly required for two's complement.
- ☐ c. An XOR gate array can invert operand B while supplying a carry-in ($C_{in} = 1$) to perform subtraction.
- ☐ d. Floating-point division hardware requires fewer logic gates than integer addition hardware.

Question 3

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Early RISC design philosophy: area minimization vs. hardware division unit tradeoffs in early embedded cores (e.g., ARM7TDMI).

How did early RISC processors, such as the ARM7TDMI, perform integer division without a dedicated hardware divider?

- a. Through specialized floating-point coprocessor registers
- b. By triggering pipeline flushing on each arithmetic call
- c. Using compiler-generated software loops of simpler shift and subtract instructions
- d. By mapping dividend memory directly into ROM lookup tables

Question 4

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: RISC silicon area economy: understanding architectural reasons for omitting hardware division units.

Why did early embedded RISC architectures avoid implementing dedicated integer division circuitry directly in silicon?

- a. Hardware division is rarely used in real software compared to add/sub, and the circuitry consumed significant silicon area and power
- b. Division operations cannot mathematically be implemented using logic gates
- c. Hardware division circuits cannot run synchronously with the CPU clock signal
- d. Division is inherently non-deterministic and prevents instruction fetching

Question 5

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Algorithmic hardware arithmetic: matching binary calculation techniques to their fundamental mathematical operations.

The classic **shift-and-add** algorithmic method is implemented in CPU datapaths to perform:

- a. Dynamic branch prediction
- b. Binary Multiplication
- c. Floating-point square root computation
- d. Modulo residue reduction

Question 6

Mark 2.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Principles of binary multiplication: bit inspection, partial product accumulation, and left/right shift scaling.

Which of the following statements regarding the binary **shift-and-add multiplication** process are **TRUE**? (Select all that apply.)

- [ ] a. If the least significant bit (LSB) of the multiplier is 1, the multiplicand is added to the running product.
- [ ] b. If the LSB of the multiplier is 0, addition is skipped for that step.
- [ ] c. The multiplicand is shifted left (or the product is shifted right) each iteration to account for the next bit weight.
- [ ] d. Multiplying two n-bit integers produces an output that can occupy up to $2n$ bits.

Question 7

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Hardware division algorithms: recognizing classic iterative methods that test and conditionally restore remainder values.

Which classic iterative algorithm is widely utilized in simple CPU hardware to execute integer division?

- ◯ a. Restoring division
- ◯ b. Matrix inversion
- ◯ c. Fast Fourier Transform
- ◯ d. Shift-and-move iteration

Question 8

Mark 2.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Mechanics of restoring division: remainder subtraction, condition checking, and restoration upon negative intermediate results.

In the **restoring division algorithm**, what occurs when subtracting the divisor from the current remainder yields a negative result? (Select all that apply.)

- [ ] a. The corresponding quotient bit is set to 0.
- [ ] b. The divisor is added back to restore the previous remainder value.
- [ ] c. The corresponding quotient bit is set to 1.
- [ ] d. The operation immediately raises a hardware divide-by-zero trap.

PART 2: PIPELINING FUNDAMENTALS & PIPELINE HAZARDS

Question 9

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Pipelining concept model: industrial operational analogy representing overlapping stages of processing.

Instruction pipelining in CPU microarchitecture is conceptually most similar to:

- ◯ a. An assembly line in a manufacturing factory
- ◯ b. A depth-first graph search algorithm
- ◯ c. A recursive function call stack
- ◯ d. Random-access memory paging

Question 10

Mark 2.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Architectural metrics of pipelining: instruction latency vs throughput, and ideal clock cycle targets.

Which of the following statements regarding the performance impact of instruction pipelining are **TRUE**? (Select all that apply.)

- ☐ a. Pipelining increases instruction throughput (instructions completed per second).
- ☐ b. Pipelining reduces the total execution latency for an isolated single instruction.
- ☐ c. In a balanced k-stage pipeline without hazards, the ideal speedup over an unpipelined CPU is approximately k.
- ☐ d. Clock cycle duration in a pipelined CPU is determined by the latency of the slowest pipeline stage plus register overhead.

Question 11

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Standard hazard classification: distinguishing legitimate pipeline data hazard acronyms from general memory components.

Which of the following is **NOT** a standard category of pipeline data hazard?

- ◯ a. RAW (Read After Write)
- ◯ b. WAR (Write After Read)
- ◯ c. RAM (Random Access Memory)
- ◯ d. WAW (Write After Write)

Question 12

Mark 2.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Taxonomy of pipeline hazards: separating structural, data, and control hazards by root physical cause.

Which of the following correctly classify the three primary types of pipeline hazards? (Select all that apply.)

- [ ] a. **Data Hazard:** Occurs when an instruction depends on the result of a previous instruction still moving through the pipeline.
- [ ] b. **Control Hazard:** Occurs when a branch decision determines the next instruction to fetch, disrupting linear sequencing.
- [ ] c. **Structural Hazard:** Occurs when hardware resource conflicts prevent two instructions from executing simultaneously.
- [ ] d. **Memory Hazard:** Occurs whenever cache lines exceed 64 bytes.

Question 13

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Branch impact on the PC: identifying the hazard resulting from changes to instruction fetch addresses.

Which type of hazard occurs when a conditional branch or jump instruction alters program flow and disrupts the pipeline instruction fetch sequence?

- ◯ a. Structural Hazard
- ◯ b. Control Hazard (Branch Hazard)
- ◯ c. WAR Hazard
- ◯ d. WAW Hazard

Question 14

Mark 2.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Methods to resolve control hazards: branch prediction, delayed branch slots, and pipeline flushes.

Which of the following techniques are commonly utilized by microprocessors to mitigate or resolve **control hazards**? (Select all that apply.)

- [ ] a. Dynamic branch prediction (predict taken vs. not taken)
- [ ] b. Branch delay slots (architectural branch delay instructions)
- [ ] c. Pipeline flushing (discarding speculatively fetched instructions upon a misprediction)
- [ ] d. Replacing all registers with dual-ported SRAM

Question 15

Mark 4.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Dependency analysis: identifying true data dependencies (RAW) where instruction 2 reads a register written by instruction 1.

Consider the following pair of sequential instructions:

```
add $s0, $t1, $t2 # Instruction 1
sub $t3, $s0, $t4 # Instruction 2
```

What type of data hazard can occur between Instruction 1 and Instruction 2?

- a. WAR (Write After Read)
- b. RAW (Read After Write)
- c. WAW (Write After Write)
- d. RAR (Read After Read)

Question 16

Mark 4.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Output dependency analysis: identifying hazards that occur when two instructions write to the same destination register.

Consider the following code sequence executing in a superscalar or out-of-order processor:

```
mul $t0, $s1, $s2 # Instruction 1 (takes 4 cycles to complete)
add $t0, $s3, $s4 # Instruction 2 (takes 1 cycle to complete)
```

If Instruction 2 finishes and writes to `$t0` before Instruction 1 overwrites it, which data hazard occurs?

- a. RAW (Read After Write)
- b. WAR (Write After Read)
- c. WAW (Write After Write / Output Dependency)
- d. Control Hazard

Question 17

Mark 6.00 out of 100.00

[FILL IN THE BLANK - ARCHITECTURAL TIMING]

Technique / Concept Hint: Pipeline stage timing of RAW hazards: Instruction 1 generates result at WB, while Instruction 2 demands operand at ID.

Analyze the exact stage timing for a RAW hazard between `add $s0, $t1, $t2` and `sub $t3, $s0, $t4`:

• Instruction 1 writes the updated result into `$s0` during its: _____ stage.

• Instruction 2 attempts to read the value of `$s0` during its: _____ stage.

• Without hardware forwarding, Instruction 2 reads a stale value because Decode occurs _____

(before / after) the Write Back of Instruction 1.

Question 18

Mark 4.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Hardware forwarding / bypassing: routing ALU outputs directly to subsequent ALU inputs without waiting for register write-back.

Which of the following statements regarding **hardware forwarding (bypassing)** are **TRUE**? (Select all that apply.)

- [ ] a. Forwarding routes the computed ALU result directly from the `EX/MEM` or `MEM/WB` pipeline registers back to the ALU multiplexers.
- [ ] b. Forwarding eliminates stalls for back-to-back R-type arithmetic dependencies (e.g., `add` followed immediately by `sub`).
- [ ] c. Forwarding completely eliminates all stalls for a `lw` instruction followed immediately by a dependent arithmetic instruction without needing pipeline bubbles.
- [ ] d. The Forwarding Unit detects dependencies by comparing destination register numbers (`Rd` or `Rt`) with source register numbers (`Rs` or `Rt`).

Question 19

Mark 6.00 out of 100.00

[FILL IN THE BLANK - PIPELINE HAZARD ANALYSIS]

Technique / Concept Hint: Out-of-order execution hazard analysis: anti-dependencies (WAR) occurring when an instruction writes before an earlier instruction reads.

Consider the instruction sequence:

```
lw $t0, 0($s0) # Instruction 1
add $t1, $t0, $s1 # Instruction 2
add $t0, $s2, $s3 # Instruction 3
```

In a modern out-of-order execution engine:

- If Instruction 3 writes to `$t0` before Instruction 2 reads from `$t0`, this produces a: _____ hazard.
- In a simple strictly in-order MIPS pipeline, does this hazard typically occur? (Yes / No): _____

Question 20

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Mitigating name dependencies (WAR and WAW) in out-of-order processors: dynamic register renaming.

What advanced hardware technique is used in modern superscalar and out-of-order CPUs to eliminate false data hazards (WAR and WAW) caused by register name reuse?

- ◯ a. Register Renaming
- ◯ b. Cache coherence sniffing
- ◯ c. Branch delay slot padding
- ◯ d. Instruction fetch throttling

PART 3: LANGUAGES, TOOLCHAINS & REGISTER ARCHITECTURE

Question 21

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Computer language abstraction levels: symbolic assembly vs. raw machine binary.

Which of the following statements best describes the relationship between assembly language and machine language?

- a. Assembly language is entirely architecture-independent, whereas machine language is hardware-specific
- b. Machine language is binary bit strings, whereas assembly language uses human-readable symbolic representations of those instructions
- c. Machine language is higher-level and requires an interpreter to execute
- d. Assembly language is executed natively by silicon without translation

Question 22

Mark 2.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Software compilation chain: distinct roles of the preprocessor, compiler, assembler, and linker.

Which of the following statements regarding the compilation and translation toolchain are **TRUE**? (Select all that apply.)

- [ ] a. The **compiler** translates high-level code (e.g., C/C++) into assembly language.
- [ ] b. The **assembler** translates assembly mnemonics into relocatable machine code object files.
- [ ] c. The **linker** combines multiple object files and libraries into a single executable binary.
- [ ] d. An assembler can execute C code directly without compilation.

Question 23

Mark 6.00 out of 100.00

[FILL IN THE BLANK - REGISTER CONVENTIONS]

Technique / Concept Hint: Standard MIPS calling conventions: assigning specific registers to argument passing, function return values, and temporaries.

State the designated role of the following standard MIPS registers according to the MIPS ABI:

- `$t0 – $t9` : _____ (Temporary registers, caller-saved / Saved registers)
- `$a0 – $a3` : _____ (Function / Syscall arguments)
- `$v0 – $v1` : _____ (Function return values / Syscall codes)

Question 24

Mark 3.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: MIPS ABI conventions: callee-saved registers (`$s0-$s7`), stack pointer (`$sp`), return address (`$ra`), and constant zero (`$zero`).

Which of the following statements regarding MIPS architectural register conventions are **TRUE**? (Select all that apply.)

- [ ] a. Registers `$s0 –$s7` are callee-saved registers whose values must be preserved across function calls.
- [ ] b. Register `$ra` ($31) holds the return address used by the `jr $ra` instruction.
- [ ] c. Register `$zero` ($0) can be overwritten with any temporary integer value.
- [ ] d. Register `$sp` ($29) serves as the hardware stack pointer.

PART 4: ASSEMBLY SHIFT-AND-ADD ALGORITHM TRACING

Question 25: Shift-and-Add Execution Trace (R1 = 12, R2 = 20)

Mark 20.00 out of 100.00

[FILL IN THE BLANK - MACHINE STATE TRACING]

Technique / Concept Hint: Binary multiplication trace: inspect LSB of multiplier (R2); determine whether addition executes or skips; scale multiplicand (R1) and multiplier (R2).

Examine the following ARM assembly routine implementing shift-and-add multiplication:

```
MOV R1, #12 @ Multiplicand = 12
MOV R2, #20 @ Multiplier = 20
MOV R0, #0 @ Accumulator Result = 0
MOV R3, #32 @ Counter = 32 bits
mult_loop:
  ANDS R4, R2, #1 @ Check LSB of Multiplier (R2)
  BEQ no_add @ If LSB == 0, skip adding
  ADD R0, R0, R1 @ If LSB == 1, add Multiplicand to Result
no_add:
  LSL R1, R1, #1 @ Double multiplicand
  LSR R2, R2, #1 @ Shift multiplier right
  SUBS R3, R3, #1 @ Decrement counter
  BNE mult_loop @ Repeat until 32 bits processed
```

Answer the following trace questions for the first iteration:

Q1: Multiplier (R2 = 20) in 32-bit binary (fill in each 4-bit block):

Q2: What is the LSB of the Multiplier (R2) during iteration 1? ____

Q3: Because of the LSB, does `ADD R0, R0, R1` execute or skip? ____

(Executes / Skips)

Q4: What instruction is executed immediately after evaluating the branch? ____

Question 26: Shift-and-Add Execution Trace (R1 = 15, R2 = 13)

Mark 15.00 out of 100.00

[FILL IN THE BLANK - MACHINE STATE TRACING]

Technique / Concept Hint: Non-zero LSB execution trace: multiplier with odd parity triggers immediate addition to accumulator R0.

Now consider the same algorithm initialized with `R1 = 15` (Multiplicand) and `R2 = 13` (Multiplier):

Q1: Multiplier (R2 = 13) represented in the lowest 8 bits of binary: _____ 2

Q2: What is the LSB of Multiplier R2 on iteration 1? _____

Q3: What will be the value in `R0` after iteration 1 finishes? _____

Q4: What will be the final value in `R0` in decimal when the entire loop completes? _____

Question 27

Mark 2.00 out of 100.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: ARM bit testing mechanics: ANDS updates condition flags (setting Z = 1 if bit is 0) without permanently altering the source register.

In the instruction `ANDS R4, R2, #1`, what is the role of the suffix S?

- ◯ a. It specifies that the operation operates in supervisor CPU mode
- ◯ b. It forces the ALU to update the CPSR condition flags (including the Zero flag Z) based on the result
- ◯ c. It forces signed arithmetic interpretation of register R2
- ◯ d. It treats R4 as a 16-bit short integer

Question 28

Mark 3.00 out of 100.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Shift logic roles: shifting multiplicand left doubles its value; shifting multiplier right exposes the next bit.

Which of the following statements correctly explain the role of LSL and LSR in the shift-and-add algorithm? (Select all that apply.)

- ☐ a. `LSL R1, R1, #1` shifts the multiplicand left by 1 bit, doubling its value for the next power-of-two position.
- ☐ b. `LSR R2, R2, #1` shifts the multiplier right by 1 bit, positioning its next bit into the LSB location.
- ☐ c. `LSL` and `LSR` are purely decorative and can be removed without altering the final product.
- ☐ d. Shifting both registers aligns the arithmetic so that partial products are weighted accurately.

Instructor Answer Key & Marking Criteria

Maximum Marks: 100.00

Quiz #4 (Unit 4 - Extended) - Comprehensive Solutions & Rationales

---

Question 1 (2.00 Marks) — Multiple Choice

Correct Answer: b. ADD and SUB

Explanation: Adders and subtractors are the elemental arithmetic components hardwired directly into the ALU core[cite: 6]. Multiplication and division are higher-order operations historically synthesized via iterative microcode or specialized functional units[cite: 6].

Question 2 (2.00 Marks) — Select All That Apply

Correct Answers: a, c

Explanation:

- **a is TRUE:** Two's complement subtraction $A - B = A + (\sim B + 1)$.
- **b is FALSE:** Dedicated subtractors are unnecessary; adders are reused.
- **c is TRUE:** Passing B through an XOR gate with an invert control line ($C_{in} = 1$) provides inversion and $+1$ increment in one circuit.
- **d is FALSE:** Floating-point dividers require vastly more gates than integer adders.

Question 3 (2.00 Marks) — Multiple Choice

Correct Answer: c. compiler loops of simpler instructions

Explanation: Early RISC chips like the ARM7TDMI omitted dedicated hardware division units to keep chip size and power low, relying on compiler runtime libraries or assembly loops[cite: 6].

Question 4 (2.00 Marks) — Multiple Choice

Correct Answer: a. Hardware division is rarely used in real software compared to add/sub, and the circuitry consumed significant silicon area and power

Explanation: In accordance with RISC design philosophy, silicon was prioritized for high-frequency common instructions (Add, Sub, Load, Store); division was infrequent enough to handle in software[cite: 6].

Question 5 (2.00 Marks) — Multiple Choice

Correct Answer: b. Binary Multiplication

Explanation: Shift-and-add algorithms accumulate shifted versions of the multiplicand based on multiplier bit values to perform multiplication[cite: 6].

Question 6 (2.00 Marks) — Select All That Apply

Correct Answers: a, b, c, d

Explanation: All four statements are mathematically and architecturally correct: LSB=1 triggers addition[cite: 6], LSB=0 skips it[cite: 6], shifting scales positional weights[cite: 6], and the product of two n-bit integers spans up to $2n$ bits.

Question 7 (2.00 Marks) — Multiple Choice

Correct Answer: a. Restoring division

Explanation: Restoring division is the standard canonical division algorithm implemented in elementary digital logic[cite: 6].

Question 8 (2.00 Marks) — Select All That Apply

Correct Answers: a, b

Explanation: In restoring division, if $(Remainder - Divisor) < 0$, the quotient bit is 0 and the divisor is added back (restored) to the remainder.

Question 9 (2.00 Marks) — Multiple Choice

Correct Answer: a. An assembly line in a manufacturing factory

Explanation: Like an assembly line, pipelining divides processing into discrete stages so multiple instructions can be serviced concurrently[cite: 6].

Question 10 (2.00 Marks) — Select All That Apply

Correct Answers: a, c, d

Explanation: Pipelining improves instruction throughput[cite: 6] with theoretical speedup $\approx k$, bounded by the slowest stage delay. It does not reduce latency for individual instructions (**b is FALSE**).

Question 11 (2.00 Marks) — Multiple Choice

Correct Answer: c. RAM

Explanation: RAM is a hardware physical memory device, not a pipeline hazard classification[cite: 6]. The valid data hazard classes are RAW, WAR, and WAW[cite: 6].

Question 12 (2.00 Marks) — Select All That Apply

Correct Answers: a, b, c

Explanation: The classical taxonomy divides pipeline hazards into Data, Control, and Structural hazards[cite: 6]. "Memory hazard" is not a canonical primary category.

Question 13 (2.00 Marks) — Multiple Choice

Correct Answer: b. Control Hazard (Branch Hazard)

Explanation: Altering the Program Counter (PC) based on branch outcomes creates control hazards because the fetch stage cannot determine the correct next instruction immediately[cite: 6].

Question 14 (2.00 Marks) — Select All That Apply

Correct Answers: a, b, c

Explanation: Branch prediction, architectural delay slots, and pipeline flushing are primary techniques for managing control hazards. Replacing registers with dual-ported SRAM resolves structural hazards, not control hazards.

Question 15 (4.00 Marks) — Multiple Choice

Correct Answer: b. RAW (Read After Write)

Explanation: Instruction 1 writes to destination `$s0`, and Instruction 2 immediately reads `$s0` as a source[cite: 6]. This is a true data dependency (RAW)[cite: 6].

Question 16 (4.00 Marks) — Multiple Choice

Correct Answer: c. WAW (Write After Write / Output Dependency)

Explanation: Both instructions write to the same register `$t0`. If Instruction 2 finishes first, Instruction 1's subsequent write-back would overwrite the newer value, causing an output hazard (WAW)[cite: 6].

Question 17 (6.00 Marks) — Fill in the Blank

Correct Answers: Write Back (WB) | Instruction Decode (ID) | before

Explanation: Instruction 1 commits its result in Write Back (WB)[cite: 6]. Instruction 2 attempts to read the register during Instruction Decode (ID)[cite: 6]. In execution order, Instruction 2's ID occurs before Instruction 1 reaches WB, causing a RAW hazard[cite: 6].

Question 18 (4.00 Marks) — Select All That Apply

Correct Answers: a, b, d

Explanation: Forwarding routes values from pipeline registers (a) and eliminates ALU-to-ALU stalls (b) using register index comparisons (d)[cite: 7]. However, forwarding cannot eliminate the load-use stall (`lw` followed immediately by dependent ALU) because memory data is not available until the end of MEM (**c is FALSE**).

Question 19 (6.00 Marks) — Fill in the Blank

Correct Answers: WAR (Write After Read) | No

Explanation: In an out-of-order CPU, if Instruction 3 writes before Instruction 2 completes reading, a WAR hazard occurs[cite: 6]. In a strictly in-order MIPS pipeline, register reads always happen in ID and register writes in WB, so WAR hazards cannot happen[cite: 6].

Question 20 (2.00 Marks) — Multiple Choice

Correct Answer: a. Register Renaming

Explanation: Register Renaming maps architectural register names onto a larger pool of physical registers, completely removing false name dependencies (WAR and WAW).

Question 21 (2.00 Marks) — Multiple Choice

Correct Answer: b. Machine language is binary bit strings, whereas assembly language uses human-readable symbolic representations of those instructions

Explanation: Assembly provides symbolic mnemonics (like `ADD`, `R1`) corresponding directly to the underlying numeric binary machine instructions[cite: 6].

Question 22 (2.00 Marks) — Select All That Apply

Correct Answers: a, b, c

Explanation: Compilers convert high-level code to assembly (a), assemblers generate machine object code (b), and linkers combine object files into executables (c). Assemblers do not directly execute high-level languages (**d is FALSE**).

Question 23 (6.00 Marks) — Fill in the Blank

Correct Answers: Temporary registers (caller-saved) | Function/syscall arguments | Return values / Syscall codes

Explanation: In MIPS ABI: `$t0–$t9` are temporaries[cite: 6], `$a0–$a3` pass arguments[cite: 6], and `$v0–$v1` return function values or specify syscall operation codes[cite: 6].

Question 24 (3.00 Marks) — Select All That Apply

Correct Answers: a, b, d

Explanation: `$s0–$s7` are callee-saved, `$ra` stores return addresses, and `$sp` points to the stack. Register `$zero` is hardwired to 0 and cannot be overwritten (**c is FALSE**).

Question 25 (20.00 Marks) — Code Tracing (R1 = 12, R2 = 20)

Q1: [0000] [0000] [0000] [0000] [0000] [0000] [0001] [0100]
Q2: LSB = [0]
Q3: [Skips] (Branches to no_add)
Q4: [LSL R1, R1, #1]

Explanation: Multiplier 20 in binary is `0001 0100`[cite: 6]. Because its LSB is 0, the ANDS test sets the Zero flag ($Z = 1$), causing `BEQ no_add` to take the branch, skipping `ADD` and jumping directly to `LSL R1, R1, #1`[cite: 6].

Question 26 (15.00 Marks) — Code Tracing (R1 = 15, R2 = 13)

Q1: [00001101]
Q2: LSB = [1]
Q3: R0 = [15]
Q4: Final R0 = [195]

Explanation: 13 in binary is `00001101`. Its LSB is 1, so BEQ does not branch. `ADD R0, R0, R1` executes, adding 15 into R0 ($0 + 15 = 15$). Upon full loop termination, $15 \times 13 = 195$.

Question 27 (2.00 Marks) — Multiple Choice

Correct Answer: b. It forces the ALU to update the CPSR condition flags (including the Zero flag Z) based on the result

Explanation: In ARM, the S suffix directs the instruction to update the Current Program Status Register (CPSR) condition flags (N, Z, C, V) according to the computed result[cite: 6].

Question 28 (3.00 Marks) — Select All That Apply

Correct Answers: a, b, d

Explanation: LSL doubles the multiplicand to align with powers of 2[cite: 6], while LSR brings subsequent multiplier bits into position[cite: 6]. Together they calculate partial products.
